

REVISA — Pack Operacional Backend e Monorepo

1. Objetivo

Este documento fecha a próxima camada operacional do projeto com artefatos textuais diretamente convertíveis em código e arquivos de repositório.

Escopo coberto:

- migrations faltantes dos schemas `polo`, `events`, `workflow`, `governance` e `analytics`
- stubs reais de `routers`, `services`, `repositories` e `models` do backend
- `permissions_seed.sql` completo
- primeiros arquivos canônicos do monorepo por domínio

A orientação continua sendo produzir um backend monolítico modular, com domínio isolado por pasta, autorização centralizada e auditoria obrigatória.

2. Migrations faltantes

2.1 `0004_create_polo_schema.py`

```
"""create polo schema

Revision ID: 0004_create_polo_schema
Revises: 0003_create_territory_schema
Create Date: 2026-04-10 00:00:04
"""

from alembic import op
import sqlalchemy as sa
from sqlalchemy.dialects import postgresql

revision = "0004_create_polo_schema"
down_revision = "0003_create_territory_schema"
branch_labels = None
depends_on = None

def upgrade() -> None:
    op.execute("CREATE SCHEMA IF NOT EXISTS polo")

    op.create_table(
        "units",
        sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
```

```

        sa.Column("organization_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("core.organizations.id"), nullable=False, unique=True),
        sa.Column("code", sa.String(length=50), nullable=True),
        sa.Column("address_label", sa.String(length=255), nullable=True),
        sa.Column("active", sa.Boolean(), nullable=False,
server_default=sa.text("true")),
        sa.Column("created_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
        schema="polo",
    )

    op.create_table(
        "beneficiarios",
        sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
        sa.Column("polo_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("polo.units.id"), nullable=False),
        sa.Column("person_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("core.persons.id"), nullable=False),
        sa.Column("source_capture_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("territory.contacts_capture.id"), nullable=True),
        sa.Column("status", sa.String(length=30), nullable=False,
server_default="PRE_CADASTRADO"),
        sa.Column("admitted_at", sa.DateTime(), nullable=True),
        sa.Column("discharged_at", sa.DateTime(), nullable=True),
        sa.Column("created_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
        sa.UniqueConstraint("polo_id", "person_id",
name="uq_polo_beneficiarios_polo_person"),
        schema="polo",
    )

    op.create_table(
        "modalidades",
        sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
        sa.Column("polo_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("polo.units.id"), nullable=False),
        sa.Column("name", sa.String(length=255), nullable=False),
        sa.Column("description", sa.Text(), nullable=True),
        sa.Column("active", sa.Boolean(), nullable=False,
server_default=sa.text("true")),
        sa.Column("created_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
        schema="polo",
    )

    op.create_table(
        "matriculas_modalidade",
        sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
        sa.Column("beneficiario_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("polo.beneficiarios.id", ondelete="CASCADE"), nullable=False),
        sa.Column("modalidade_id", postgresql.UUID(as_uuid=True),

```

```

sa.ForeignKey("polo.modalidades.id"), nullable=False),
    sa.Column("status", sa.String(length=30), nullable=False,
server_default="ATIVA"),
    sa.Column("start_date", sa.Date(), nullable=False),
    sa.Column("end_date", sa.Date(), nullable=True),
    sa.Column("created_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
    schema="polo",
)

op.create_table(
    "frecuencias",
    sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
    sa.Column("beneficiario_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("polo.beneficiarios.id", ondelete="CASCADE"), nullable=False),
    sa.Column("modalidade_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("polo.modalidades.id"), nullable=True),
    sa.Column("registered_by_user_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("iam.users.id"), nullable=False),
    sa.Column("activity_date", sa.Date(), nullable=False),
    sa.Column("present", sa.Boolean(), nullable=False),
    sa.Column("notes", sa.Text(), nullable=True),
    sa.Column("created_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
    sa.UniqueConstraint("beneficiario_id", "modalidade_id",
"activity_date", name="uq_polo_frecuencias_unique"),
    schema="polo",
)

op.create_table(
    "ocorrencias",
    sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
    sa.Column("polo_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("polo.units.id"), nullable=False),
    sa.Column("beneficiario_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("polo.beneficiarios.id"), nullable=True),
    sa.Column("registered_by_user_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("iam.users.id"), nullable=False),
    sa.Column("severity", sa.String(length=20), nullable=False),
    sa.Column("title", sa.String(length=255), nullable=False),
    sa.Column("description", sa.Text(), nullable=False),
    sa.Column("status", sa.String(length=20), nullable=False,
server_default="OPEN"),
    sa.Column("created_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
    sa.Column("updated_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
    schema="polo",
)

op.create_table(

```

```

        "daily_logs",
        sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
        sa.Column("polo_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("polo.units.id"), nullable=False),
        sa.Column("created_by_user_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("iam.users.id"), nullable=False),
        sa.Column("log_date", sa.Date(), nullable=False),
        sa.Column("content", sa.Text(), nullable=False),
        sa.Column("created_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
        sa.UniqueConstraint("polo_id", "log_date", "created_by_user_id",
name="uq_polo_daily_logs_unique"),
        schema="polo",
    )

    op.create_table(
        "purchase_requests",
        sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
        sa.Column("polo_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("polo.units.id"), nullable=False),
        sa.Column("requested_by_user_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("iam.users.id"), nullable=False),
        sa.Column("title", sa.String(length=255), nullable=False),
        sa.Column("description", sa.Text(), nullable=True),
        sa.Column("status", sa.String(length=20), nullable=False,
server_default="OPEN"),
        sa.Column("priority", sa.String(length=20), nullable=False,
server_default="MEDIUM"),
        sa.Column("created_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
        sa.Column("updated_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
        schema="polo",
    )

def downgrade() -> None:
    op.drop_table("purchase_requests", schema="polo")
    op.drop_table("daily_logs", schema="polo")
    op.drop_table("ocorrencias", schema="polo")
    op.drop_table("frequencias", schema="polo")
    op.drop_table("matriculas_modalidade", schema="polo")
    op.drop_table("modalidades", schema="polo")
    op.drop_table("beneficiarios", schema="polo")
    op.drop_table("units", schema="polo")
    op.execute("DROP SCHEMA IF EXISTS polo CASCADE")

```

2.2 0005_create_events_schema.py

```
"""create events schema

Revision ID: 0005_create_events_schema
Revises: 0004_create_polo_schema
Create Date: 2026-04-10 00:00:05
"""

from alembic import op
import sqlalchemy as sa
from sqlalchemy.dialects import postgresql

revision = "0005_create_events_schema"
down_revision = "0004_create_polo_schema"
branch_labels = None
depends_on = None

def upgrade() -> None:
    op.execute("CREATE SCHEMA IF NOT EXISTS events")

    op.create_table(
        "events",
        sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
        sa.Column("organization_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("core.organizations.id"), nullable=True),
        sa.Column("vereador_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("core.vereadores.id"), nullable=True),
        sa.Column("created_by_user_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("iam.users.id"), nullable=False),
        sa.Column("title", sa.String(length=255), nullable=False),
        sa.Column("description", sa.Text(), nullable=True),
        sa.Column("event_type", sa.String(length=40), nullable=False),
        sa.Column("status", sa.String(length=20), nullable=False,
server_default="PLANNED"),
        sa.Column("start_at", sa.DateTime(), nullable=False),
        sa.Column("end_at", sa.DateTime(), nullable=True),
        sa.Column("created_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
        sa.Column("updated_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
        schema="events",
    )

    op.create_table(
        "activities",
        sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
        sa.Column("event_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("events.events.id", ondelete="CASCADE"), nullable=True),
```

```

        sa.Column("polo_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("polo.units.id"), nullable=True),
        sa.Column("organization_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("core.organizations.id"), nullable=True),
        sa.Column("title", sa.String(length=255), nullable=False),
        sa.Column("activity_type", sa.String(length=40), nullable=False),
        sa.Column("starts_at", sa.DateTime(), nullable=False),
        sa.Column("ends_at", sa.DateTime(), nullable=True),
        sa.Column("recurrence_rule", sa.String(length=255), nullable=True),
        sa.Column("created_by_user_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("iam.users.id"), nullable=False),
        sa.Column("created_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
        schema="events",
    )

    op.create_table(
        "participations",
        sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
        sa.Column("activity_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("events.activities.id", ondelete="CASCADE"), nullable=False),
        sa.Column("person_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("core.persons.id"), nullable=False),
        sa.Column("registered_by_user_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("iam.users.id"), nullable=False),
        sa.Column("checkin_at", sa.DateTime(), nullable=True),
        sa.Column("status", sa.String(length=20), nullable=False,
server_default="REGISTERED"),
        sa.Column("notes", sa.Text(), nullable=True),
        sa.Column("created_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
        sa.UniqueConstraint("activity_id", "person_id",
name="uq_events_participations_activity_person"),
        schema="events",
    )

def downgrade() -> None:
    op.drop_table("participations", schema="events")
    op.drop_table("activities", schema="events")
    op.drop_table("events", schema="events")
    op.execute("DROP SCHEMA IF EXISTS events CASCADE")

```

2.3 0006_create_workflow_schema.py

```

"""create workflow schema

Revision ID: 0006_create_workflow_schema
Revises: 0005_create_events_schema
Create Date: 2026-04-10 00:00:06

```

"""

```
from alembic import op
import sqlalchemy as sa
from sqlalchemy.dialects import postgresql
```

```
revision = "0006_create_workflow_schema"
down_revision = "0005_create_events_schema"
branch_labels = None
depends_on = None
```

```
def upgrade() -> None:
```

```
    op.execute("CREATE SCHEMA IF NOT EXISTS workflow")
```

```
    op.create_table(
        "tasks",
        sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
        sa.Column("organization_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("core.organizations.id"), nullable=True),
        sa.Column("vereador_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("core.vereadores.id"), nullable=True),
        sa.Column("polo_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("polo.units.id"), nullable=True),
        sa.Column("person_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("core.persons.id"), nullable=True),
        sa.Column("demand_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("territory.demands.id"), nullable=True),
        sa.Column("assigned_to_user_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("iam.users.id"), nullable=True),
        sa.Column("created_by_user_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("iam.users.id"), nullable=False),
        sa.Column("task_type", sa.String(length=50), nullable=False),
        sa.Column("title", sa.String(length=255), nullable=False),
        sa.Column("description", sa.Text(), nullable=True),
        sa.Column("priority", sa.String(length=20), nullable=False,
server_default="MEDIUM"),
        sa.Column("status", sa.String(length=20), nullable=False,
server_default="OPEN"),
        sa.Column("due_at", sa.DateTime(), nullable=True),
        sa.Column("completed_at", sa.DateTime(), nullable=True),
        sa.Column("created_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
        sa.Column("updated_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
        schema="workflow",
    )
```

```
    op.create_table(
        "task_outcomes",
        sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
```

```

        sa.Column("task_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("workflow.tasks.id", ondelete="CASCADE"), nullable=False),
        sa.Column("outcome_type", sa.String(length=50), nullable=False),
        sa.Column("notes", sa.Text(), nullable=True),
        sa.Column("recorded_by_user_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("iam.users.id"), nullable=False),
        sa.Column("recorded_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
        schema="workflow",
    )

def downgrade() -> None:
    op.drop_table("task_outcomes", schema="workflow")
    op.drop_table("tasks", schema="workflow")
    op.execute("DROP SCHEMA IF EXISTS workflow CASCADE")

```

2.4 0007_create_governance_schema.py

```

"""create governance schema

Revision ID: 0007_create_governance_schema
Revises: 0006_create_workflow_schema
Create Date: 2026-04-10 00:00:07
"""

from alembic import op
import sqlalchemy as sa
from sqlalchemy.dialects import postgresql

revision = "0007_create_governance_schema"
down_revision = "0006_create_workflow_schema"
branch_labels = None
depends_on = None

def upgrade() -> None:
    op.execute("CREATE SCHEMA IF NOT EXISTS governance")

    op.create_table(
        "audit_logs",
        sa.Column("id", sa.BigInteger(), primary_key=True,
autoincrement=True),
        sa.Column("user_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("iam.users.id"), nullable=True),
        sa.Column("action", sa.String(length=120), nullable=False),
        sa.Column("entity_schema", sa.String(length=60), nullable=False),
        sa.Column("entity_name", sa.String(length=120), nullable=False),
        sa.Column("entity_id", postgresql.UUID(as_uuid=True), nullable=True),
        sa.Column("old_values_json",

```



```

postgresql.JSONB(astext_type=sa.Text()), nullable=True),
    sa.Column("new_values_json",
postgresql.JSONB(astext_type=sa.Text()), nullable=True),
    sa.Column("ip_address", postgresql.INET(), nullable=True),
    sa.Column("user_agent", sa.Text(), nullable=True),
    sa.Column("created_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
    schema="governance",
)

op.create_table(
    "access_logs",
    sa.Column("id", sa.BigInteger(), primary_key=True,
autoincrement=True),
    sa.Column("user_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("iam.users.id"), nullable=True),
    sa.Column("event_type", sa.String(length=60), nullable=False),
    sa.Column("success", sa.Boolean(), nullable=False),
    sa.Column("ip_address", postgresql.INET(), nullable=True),
    sa.Column("user_agent", sa.Text(), nullable=True),
    sa.Column("created_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
    schema="governance",
)

op.create_table(
    "export_logs",
    sa.Column("id", sa.BigInteger(), primary_key=True,
autoincrement=True),
    sa.Column("user_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("iam.users.id"), nullable=False),
    sa.Column("export_type", sa.String(length=60), nullable=False),
    sa.Column("filter_json", postgresql.JSONB(astext_type=sa.Text()),
nullable=True),
    sa.Column("row_count", sa.Integer(), nullable=True),
    sa.Column("created_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
    schema="governance",
)

op.create_table(
    "privacy_requests",
    sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
    sa.Column("person_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("core.persons.id"), nullable=False),
    sa.Column("request_type", sa.String(length=30), nullable=False),
    sa.Column("status", sa.String(length=20), nullable=False,
server_default="OPEN"),
    sa.Column("requested_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
    sa.Column("processed_at", sa.DateTime(), nullable=True),

```

```

        sa.Column("processed_by_user_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("iam.users.id"), nullable=True),
        sa.Column("notes", sa.Text(), nullable=True),
        schema="governance",
    )

def downgrade() -> None:
    op.drop_table("privacy_requests", schema="governance")
    op.drop_table("export_logs", schema="governance")
    op.drop_table("access_logs", schema="governance")
    op.drop_table("audit_logs", schema="governance")
    op.execute("DROP SCHEMA IF EXISTS governance CASCADE")

```

2.5 0008_create_analytics_schema.py

```

"""create analytics schema

Revision ID: 0008_create_analytics_schema
Revises: 0007_create_governance_schema
Create Date: 2026-04-10 00:00:08
"""

from alembic import op
import sqlalchemy as sa
from sqlalchemy.dialects import postgresql

revision = "0008_create_analytics_schema"
down_revision = "0007_create_governance_schema"
branch_labels = None
depends_on = None

def upgrade() -> None:
    op.execute("CREATE SCHEMA IF NOT EXISTS analytics")

    op.create_table(
        "feature_store_operational",
        sa.Column("id", postgresql.UUID(as_uuid=True), primary_key=True),
        sa.Column("person_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("core.persons.id"), nullable=False),
        sa.Column("ref_date", sa.Date(), nullable=False),
        sa.Column("interactions_30d", sa.Integer(), nullable=False,
server_default="0"),
        sa.Column("activities_90d", sa.Integer(), nullable=False,
server_default="0"),
        sa.Column("open_demands_count", sa.Integer(), nullable=False,
server_default="0"),
        sa.Column("active_beneficiary", sa.Boolean(), nullable=False,
server_default=sa.text("false")),

```

```

        sa.Column("linked_vereador_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("core.vereadores.id"), nullable=True),
        sa.Column("linked_polo_id", postgresql.UUID(as_uuid=True),
sa.ForeignKey("polo.units.id"), nullable=True),
        sa.Column("created_at", sa.DateTime(), nullable=False,
server_default=sa.text("now()")),
        sa.UniqueConstraint("person_id", "ref_date",
name="uq_analytics_feature_store_person_ref_date"),
        schema="analytics",
    )

    op.execute(
        """
        create materialized view analytics.mv_vereador_dashboard as
        select
            v.id as vereador_id,
            count(distinct cc.id) as total_captures,
            count(distinct pb.id) as total_beneficiarios,
            count(distinct d.id) filter (where d.status = 'OPEN') as
open_demands,
            count(distinct t.id) filter (where t.status = 'OPEN') as
open_tasks
        from core.vereadores v
        left join territory.contacts_capture cc on cc.vereador_id = v.id
        left join territory.demands d on d.vereador_id = v.id
        left join workflow.tasks t on t.vereador_id = v.id
        left join territory.contacts_capture cc2 on cc2.vereador_id = v.id
        left join polo.beneficiarios pb on pb.source_capture_id = cc2.id
        group by v.id
        """
    )

def downgrade() -> None:
    op.execute("drop materialized view if exists
analytics.mv_vereador_dashboard")
    op.drop_table("feature_store_operational", schema="analytics")
    op.execute("DROP SCHEMA IF EXISTS analytics CASCADE")

```

3. Stubs reais do backend

3.1 Convenções dos stubs

Cada domínio terá:

- models.py
- schemas.py
- repository.py

- `service.py`
- `policy.py`
- router correspondente

Os stubs abaixo são canônicos e servem de ponto inicial para o time.

3.2 `apps/api/app/domain/polo/models.py`

```
from datetime import date, datetime
from uuid import uuid4

from sqlalchemy import Boolean, Date, DateTime, ForeignKey, String, Text,
func
from sqlalchemy.dialects.postgresql import UUID
from sqlalchemy.orm import Mapped, mapped_column

from app.core.database import Base

class PoloUnit(Base):
    __tablename__ = "units"
    __table_args__ = {"schema": "polo"}

    id: Mapped[str] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid4)
    organization_id: Mapped[str] = mapped_column(UUID(as_uuid=True),
ForeignKey("core.organizations.id"), unique=True)
    code: Mapped[str | None] = mapped_column(String(50))
    address_label: Mapped[str | None] = mapped_column(String(255))
    active: Mapped[bool] = mapped_column(Boolean, default=True,
nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime,
server_default=func.now(), nullable=False)

class PoloBeneficiario(Base):
    __tablename__ = "beneficiarios"
    __table_args__ = {"schema": "polo"}

    id: Mapped[str] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid4)
    polo_id: Mapped[str] = mapped_column(UUID(as_uuid=True),
ForeignKey("polo.units.id"), nullable=False)
    person_id: Mapped[str] = mapped_column(UUID(as_uuid=True),
ForeignKey("core.persons.id"), nullable=False)
    source_capture_id: Mapped[str | None] =
mapped_column(UUID(as_uuid=True),
ForeignKey("territory.contacts_capture.id"))
    status: Mapped[str] = mapped_column(String(30),
```

```

default="PRE_CADASTRADO", nullable=False)
    admitted_at: Mapped[datetime | None] = mapped_column(DateTime)
    discharged_at: Mapped[datetime | None] = mapped_column(DateTime)
    created_at: Mapped[datetime] = mapped_column(DateTime,
server_default=func.now(), nullable=False)

class Modalidade(Base):
    __tablename__ = "modalidades"
    __table_args__ = {"schema": "polo"}

    id: Mapped[str] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid4)
    polo_id: Mapped[str] = mapped_column(UUID(as_uuid=True),
ForeignKey("polo.units.id"), nullable=False)
    name: Mapped[str] = mapped_column(String(255), nullable=False)
    description: Mapped[str | None] = mapped_column(Text)
    active: Mapped[bool] = mapped_column(Boolean, default=True,
nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime,
server_default=func.now(), nullable=False)

class Frecuencia(Base):
    __tablename__ = "frecuencias"
    __table_args__ = {"schema": "polo"}

    id: Mapped[str] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid4)
    beneficiario_id: Mapped[str] = mapped_column(UUID(as_uuid=True),
ForeignKey("polo.beneficiarios.id"), nullable=False)
    modalidade_id: Mapped[str | None] = mapped_column(UUID(as_uuid=True),
ForeignKey("polo.modalidades.id"))
    registered_by_user_id: Mapped[str] = mapped_column(UUID(as_uuid=True),
ForeignKey("iam.users.id"), nullable=False)
    activity_date: Mapped[date] = mapped_column(Date, nullable=False)
    present: Mapped[bool] = mapped_column(Boolean, nullable=False)
    notes: Mapped[str | None] = mapped_column(Text)
    created_at: Mapped[datetime] = mapped_column(DateTime,
server_default=func.now(), nullable=False)

```

3.3 apps/api/app/domain/polo/schemas.py

```

from datetime import date
from pydantic import BaseModel, Field
from uuid import UUID

class PoloBeneficiarioCreate(BaseModel):
    person_id: UUID

```

```

source_capture_id: UUID | None = None
status: str = "PRE_CADASTRADO"

class PoloBeneficiarioOut(BaseModel):
    id: UUID
    polo_id: UUID
    person_id: UUID
    source_capture_id: UUID | None = None
    status: str

    model_config = {"from_attributes": True}

class AttendanceCreate(BaseModel):
    beneficiario_id: UUID
    modalidade_id: UUID | None = None
    activity_date: date
    present: bool
    notes: str | None = Field(default=None, max_length=2000)

class AttendanceOut(BaseModel):
    id: UUID
    beneficiario_id: UUID
    modalidade_id: UUID | None = None
    activity_date: date
    present: bool
    notes: str | None = None

    model_config = {"from_attributes": True}

```

3.4 apps/api/app/domain/polo/repository.py

```

from sqlalchemy import select
from sqlalchemy.orm import Session

from app.domain.polo.models import PoloBeneficiario, Frequencia

class PoloRepository:
    def __init__(self, db: Session):
        self.db = db

    def list_beneficiarios(self, polo_id):
        stmt = select(PoloBeneficiario).where(PoloBeneficiario.polo_id ==
        polo_id)
        return self.db.execute(stmt).scalars().all()

    def create_beneficiario(self, entity: PoloBeneficiario) ->

```

```

PoloBeneficiario:
    self.db.add(entity)
    self.db.flush()
    self.db.refresh(entity)
    return entity

def create_frequencia(self, entity: Frequencia) -> Frequencia:
    self.db.add(entity)
    self.db.flush()
    self.db.refresh(entity)
    return entity

```

3.5 apps/api/app/domain/polo/service.py

```

from app.domain.polo.models import PoloBeneficiario, Frequencia
from app.domain.polo.repository import PoloRepository
from app.domain.polo.schemas import AttendanceCreate, PoloBeneficiarioCreate

class PoloService:
    def __init__(self, repo: PoloRepository):
        self.repo = repo

    def list_beneficiarios(self, polo_id):
        return self.repo.list_beneficiarios(polo_id)

    def create_beneficiario(self, polo_id, payload: PoloBeneficiarioCreate):
        entity = PoloBeneficiario(
            polo_id=polo_id,
            person_id=payload.person_id,
            source_capture_id=payload.source_capture_id,
            status=payload.status,
        )
        return self.repo.create_beneficiario(entity)

    def register_attendance(self, registered_by_user_id, payload:
AttendanceCreate):
        entity = Frequencia(
            beneficiario_id=payload.beneficiario_id,
            modalidade_id=payload.modalidade_id,
            registered_by_user_id=registered_by_user_id,
            activity_date=payload.activity_date,
            present=payload.present,
            notes=payload.notes,
        )
        return self.repo.create_frequencia(entity)

```

3.6 apps/api/app/domain/polo/policy.py

```
class PoloPolicy:
    @staticmethod
    def can_manage_polo(current_user, polo_id) -> bool:
        if current_user.is_global_admin:
            return True
        return polo_id in current_user.allowed_polo_ids
```

3.7 apps/api/app/api/v1/routers/polos.py

```
from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session
from uuid import UUID

from app.core.database import get_db
from app.domain.polo.repository import PoloRepository
from app.domain.polo.schemas import AttendanceCreate, AttendanceOut,
PoloBeneficiarioCreate, PoloBeneficiarioOut
from app.domain.polo.service import PoloService

router = APIRouter()

@router.get("/{id}/beneficiarios", response_model=list[PoloBeneficiarioOut])
def list_beneficiarios(id: UUID, db: Session = Depends(get_db)):
    service = PoloService(PoloRepository(db))
    return service.list_beneficiarios(id)

@router.post("/{id}/beneficiarios", response_model=PoloBeneficiarioOut,
status_code=201)
def create_beneficiario(id: UUID, payload: PoloBeneficiarioCreate, db:
Session = Depends(get_db)):
    service = PoloService(PoloRepository(db))
    return service.create_beneficiario(id, payload)

@router.post("/{id}/frecuencias", response_model=AttendanceOut,
status_code=201)
def register_attendance(id: UUID, payload: AttendanceCreate, db: Session =
Depends(get_db)):
    service = PoloService(PoloRepository(db))
    return service.register_attendance(registered_by_user_id=None,
payload=payload)
```


3.8 apps/api/app/domain/territory/models.py

```
from datetime import datetime
from uuid import uuid4

from sqlalchemy import DateTime, ForeignKey, Numeric, String, Text, func
from sqlalchemy.dialects.postgresql import UUID
from sqlalchemy.orm import Mapped, mapped_column

from app.core.database import Base

class ContactCapture(Base):
    __tablename__ = "contacts_capture"
    __table_args__ = {"schema": "territory"}

    id: Mapped[str] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid4)
    captured_by_user_id: Mapped[str] = mapped_column(UUID(as_uuid=True),
ForeignKey("iam.users.id"), nullable=False)
    organization_id: Mapped[str | None] = mapped_column(UUID(as_uuid=True),
ForeignKey("core.organizations.id"))
    vereador_id: Mapped[str | None] = mapped_column(UUID(as_uuid=True),
ForeignKey("core.veredores.id"))
    team_id: Mapped[str | None] = mapped_column(UUID(as_uuid=True),
ForeignKey("core.teams.id"))
    person_id: Mapped[str | None] = mapped_column(UUID(as_uuid=True),
ForeignKey("core.persons.id"))
    origin: Mapped[str] = mapped_column(String(30), nullable=False)
    classification: Mapped[str] = mapped_column(String(30), nullable=False)
    full_name: Mapped[str] = mapped_column(String(255), nullable=False)
    phone: Mapped[str | None] = mapped_column(String(30))
    district: Mapped[str | None] = mapped_column(String(120))
    notes: Mapped[str | None] = mapped_column(Text)
    priority_level: Mapped[str | None] = mapped_column(String(20))
    capture_status: Mapped[str] = mapped_column(String(30), default="NEW",
nullable=False)
    latitude: Mapped[float | None] = mapped_column(Numeric(10, 7))
    longitude: Mapped[float | None] = mapped_column(Numeric(10, 7))
    created_at: Mapped[datetime] = mapped_column(DateTime,
server_default=func.now(), nullable=False)
    updated_at: Mapped[datetime] = mapped_column(DateTime,
server_default=func.now(), nullable=False)
```

3.9 apps/api/app/domain/territory/schemas.py

```
from pydantic import BaseModel
from uuid import UUID
```

```

class ContactCaptureCreate(BaseModel):
    origin: str
    classification: str
    full_name: str
    phone: str | None = None
    district: str | None = None
    notes: str | None = None
    vereador_id: UUID | None = None
    team_id: UUID | None = None
    latitude: float | None = None
    longitude: float | None = None

class ContactCaptureOut(BaseModel):
    id: UUID
    origin: str
    classification: str
    full_name: str
    phone: str | None = None
    district: str | None = None
    notes: str | None = None
    capture_status: str

    model_config = {"from_attributes": True}

```

3.10 apps/api/app/domain/territory/repository.py

```

from sqlalchemy import select
from sqlalchemy.orm import Session

from app.domain.territory.models import ContactCapture

class TerritoryRepository:
    def __init__(self, db: Session):
        self.db = db

    def list_captures(self):
        return self.db.execute(select(ContactCapture)).scalars().all()

    def create_capture(self, entity: ContactCapture) -> ContactCapture:
        self.db.add(entity)
        self.db.flush()
        self.db.refresh(entity)
        return entity

```

3.11 apps/api/app/domain/territory/service.py

```
from app.domain.territory.models import ContactCapture
from app.domain.territory.repository import TerritoryRepository
from app.domain.territory.schemas import ContactCaptureCreate

class TerritoryService:
    def __init__(self, repo: TerritoryRepository):
        self.repo = repo

    def list_captures(self):
        return self.repo.list_captures()

    def create_capture(self, captured_by_user_id, payload:
ContactCaptureCreate):
        entity = ContactCapture(
            captured_by_user_id=captured_by_user_id,
            vereador_id=payload.vereador_id,
            team_id=payload.team_id,
            origin=payload.origin,
            classification=payload.classification,
            full_name=payload.full_name,
            phone=payload.phone,
            district=payload.district,
            notes=payload.notes,
            latitude=payload.latitude,
            longitude=payload.longitude,
        )
        return self.repo.create_capture(entity)
```

3.12 apps/api/app/api/v1/routers/contacts_capture.py

```
from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session

from app.core.database import get_db
from app.domain.territory.repository import TerritoryRepository
from app.domain.territory.schemas import ContactCaptureCreate,
ContactCaptureOut
from app.domain.territory.service import TerritoryService

router = APIRouter()

@router.get("", response_model=list[ContactCaptureOut])
def list_captures(db: Session = Depends(get_db)):
    return TerritoryService(TerritoryRepository(db)).list_captures()
```

```

@router.post("", response_model=ContactCaptureOut, status_code=201)
def create_capture(payload: ContactCaptureCreate, db: Session =
Depends(get_db)):
    return
TerritoryService(TerritoryRepository(db)).create_capture(captured_by_user_id=None,
payload=payload)

```

3.13 apps/api/app/domain/workflow/models.py

```

from datetime import datetime
from uuid import uuid4

from sqlalchemy import DateTime, ForeignKey, String, Text, func
from sqlalchemy.dialects.postgresql import UUID
from sqlalchemy.orm import Mapped, mapped_column

from app.core.database import Base

class Task(Base):
    __tablename__ = "tasks"
    __table_args__ = {"schema": "workflow"}

    id: Mapped[str] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid4)
    organization_id: Mapped[str | None] = mapped_column(UUID(as_uuid=True),
ForeignKey("core.organizations.id"))
    vereador_id: Mapped[str | None] = mapped_column(UUID(as_uuid=True),
ForeignKey("core.veredores.id"))
    polo_id: Mapped[str | None] = mapped_column(UUID(as_uuid=True),
ForeignKey("polo.units.id"))
    person_id: Mapped[str | None] = mapped_column(UUID(as_uuid=True),
ForeignKey("core.persons.id"))
    demand_id: Mapped[str | None] = mapped_column(UUID(as_uuid=True),
ForeignKey("territory.demands.id"))
    assigned_to_user_id: Mapped[str | None] =
mapped_column(UUID(as_uuid=True), ForeignKey("iam.users.id"))
    created_by_user_id: Mapped[str] = mapped_column(UUID(as_uuid=True),
ForeignKey("iam.users.id"), nullable=False)
    task_type: Mapped[str] = mapped_column(String(50), nullable=False)
    title: Mapped[str] = mapped_column(String(255), nullable=False)
    description: Mapped[str | None] = mapped_column(Text)
    priority: Mapped[str] = mapped_column(String(20), default="MEDIUM",
nullable=False)
    status: Mapped[str] = mapped_column(String(20), default="OPEN",
nullable=False)

```

```

    due_at: Mapped[datetime | None] = mapped_column(DateTime)
    completed_at: Mapped[datetime | None] = mapped_column(DateTime)
    created_at: Mapped[datetime] = mapped_column(DateTime,
server_default=func.now(), nullable=False)
    updated_at: Mapped[datetime] = mapped_column(DateTime,
server_default=func.now(), nullable=False)

```

3.14 apps/api/app/domain/workflow/schemas.py

```

from datetime import datetime
from pydantic import BaseModel
from uuid import UUID

class TaskCreate(BaseModel):
    organization_id: UUID | None = None
    vereador_id: UUID | None = None
    polo_id: UUID | None = None
    person_id: UUID | None = None
    demand_id: UUID | None = None
    assigned_to_user_id: UUID | None = None
    task_type: str
    title: str
    description: str | None = None
    priority: str = "MEDIUM"
    due_at: datetime | None = None

class TaskOut(BaseModel):
    id: UUID
    task_type: str
    title: str
    description: str | None = None
    priority: str
    status: str
    due_at: datetime | None = None

model_config = {"from_attributes": True}

```

3.15 apps/api/app/domain/workflow/repository.py

```

from sqlalchemy import select
from sqlalchemy.orm import Session

from app.domain.workflow.models import Task

class WorkflowRepository:

```

```

def __init__(self, db: Session):
    self.db = db

def list_tasks(self):
    return self.db.execute(select(Task)).scalars().all()

def create_task(self, entity: Task) -> Task:
    self.db.add(entity)
    self.db.flush()
    self.db.refresh(entity)
    return entity

```

3.16 apps/api/app/domain/workflow/service.py

```

from app.domain.workflow.models import Task
from app.domain.workflow.repository import WorkflowRepository
from app.domain.workflow.schemas import TaskCreate

class WorkflowService:
    def __init__(self, repo: WorkflowRepository):
        self.repo = repo

    def list_tasks(self):
        return self.repo.list_tasks()

    def create_task(self, created_by_user_id, payload: TaskCreate):
        entity = Task(
            organization_id=payload.organization_id,
            vereador_id=payload.vereador_id,
            polo_id=payload.polo_id,
            person_id=payload.person_id,
            demand_id=payload.demand_id,
            assigned_to_user_id=payload.assigned_to_user_id,
            created_by_user_id=created_by_user_id,
            task_type=payload.task_type,
            title=payload.title,
            description=payload.description,
            priority=payload.priority,
            due_at=payload.due_at,
        )
        return self.repo.create_task(entity)

```

3.17 apps/api/app/api/v1/routers/tasks.py

```

from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session

```

```

from app.core.database import get_db
from app.domain.workflow.repository import WorkflowRepository
from app.domain.workflow.schemas import TaskCreate, TaskOut
from app.domain.workflow.service import WorkflowService

router = APIRouter()

@router.get("", response_model=list[TaskOut])
def list_tasks(db: Session = Depends(get_db)):
    return WorkflowService(WorkflowRepository(db)).list_tasks()

@router.post("", response_model=TaskOut, status_code=201)
def create_task(payload: TaskCreate, db: Session = Depends(get_db)):
    return
WorkflowService(WorkflowRepository(db)).create_task(created_by_user_id=None,
payload=payload)

```

3.18 apps/api/app/domain/analytics/repository.py

```

from sqlalchemy import text
from sqlalchemy.orm import Session

class AnalyticsRepository:
    def __init__(self, db: Session):
        self.db = db

    def get_vereador_dashboard(self, vereador_id):
        stmt = text(
            """
            select vereador_id, total_captures, total_beneficiarios,
open_demands, open_tasks
            from analytics.mv_vereador_dashboard
            where vereador_id = :vereador_id
            """
        )
        row = self.db.execute(stmt, {"vereador_id":
vereador_id}).mappings().first()
        return dict(row) if row else None

```

3.19 apps/api/app/domain/analytics/service.py

```

from app.domain.analytics.repository import AnalyticsRepository

```

```
class AnalyticsService:
    def __init__(self, repo: AnalyticsRepository):
        self.repo = repo

    def get_vereador_dashboard(self, vereador_id):
        return self.repo.get_vereador_dashboard(vereador_id)
```

3.20 apps/api/app/api/v1/routers/dashboards.py

```
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session
from uuid import UUID

from app.core.database import get_db
from app.domain.analytics.repository import AnalyticsRepository
from app.domain.analytics.service import AnalyticsService

router = APIRouter()

@router.get("/vereadores/{id}/dashboard")
def vereador_dashboard(id: UUID, db: Session = Depends(get_db)):
    data = AnalyticsService(AnalyticsRepository(db)).get_vereador_dashboard(id)
    if not data:
        raise HTTPException(status_code=404, detail="Dashboard não encontrado")
    return data
```

4. permissions_seed.sql completo

```
insert into iam.permissions (id, code, name) values
(gen_random_uuid(), 'auth.login', 'Realizar login'),
(gen_random_uuid(), 'user.read', 'Consultar usuários'),
(gen_random_uuid(), 'user.create', 'Criar usuários'),
(gen_random_uuid(), 'user.update', 'Atualizar usuários'),
(gen_random_uuid(), 'user.manage_roles', 'Gerir papéis de usuários'),
(gen_random_uuid(), 'user.manage_scopes', 'Gerir escopos de usuários'),
(gen_random_uuid(), 'organization.read', 'Consultar organizações'),
(gen_random_uuid(), 'organization.create', 'Criar organizações'),
(gen_random_uuid(), 'organization.update', 'Atualizar organizações'),
(gen_random_uuid(), 'vereador.read', 'Consultar vereadores'),
(gen_random_uuid(), 'vereador.create', 'Criar vereadores'),
(gen_random_uuid(), 'vereador.update', 'Atualizar vereadores'),
(gen_random_uuid(), 'team.read', 'Consultar equipes'),
```



```

(gen_random_uuid(), 'team.create', 'Criar equipes'),
(gen_random_uuid(), 'team.update', 'Atualizar equipes'),
(gen_random_uuid(), 'person.read', 'Consultar pessoas'),
(gen_random_uuid(), 'person.create', 'Criar pessoas'),
(gen_random_uuid(), 'person.update', 'Atualizar pessoas'),
(gen_random_uuid(), 'person.link', 'Vincular pessoa a contexto'),
(gen_random_uuid(), 'consent.read', 'Consultar consentimentos'),
(gen_random_uuid(), 'consent.create', 'Criar consentimentos'),
(gen_random_uuid(), 'consent.revoke', 'Revogar consentimentos'),
(gen_random_uuid(), 'capture.read', 'Consultar captações'),
(gen_random_uuid(), 'capture.create', 'Criar captações'),
(gen_random_uuid(), 'capture.classify', 'Classificar captações'),
(gen_random_uuid(), 'capture.forward', 'Encaminhar captações'),
(gen_random_uuid(), 'capture.convert', 'Converter captação em
beneficiário'),
  (gen_random_uuid(), 'polo.read', 'Consultar polos'),
  (gen_random_uuid(), 'polo.create', 'Criar polos'),
  (gen_random_uuid(), 'polo.update', 'Atualizar polos'),
  (gen_random_uuid(), 'polo.manage_beneficiary', 'Gerir beneficiários do
polo'),
  (gen_random_uuid(), 'modality.read', 'Consultar modalidades'),
  (gen_random_uuid(), 'modality.create', 'Criar modalidades'),
  (gen_random_uuid(), 'modality.update', 'Atualizar modalidades'),
  (gen_random_uuid(), 'attendance.read', 'Consultar frequências'),
  (gen_random_uuid(), 'attendance.create', 'Registrar frequências'),
  (gen_random_uuid(), 'occurrence.read', 'Consultar ocorrências'),
  (gen_random_uuid(), 'occurrence.create', 'Registrar ocorrências'),
  (gen_random_uuid(), 'daily_log.read', 'Consultar diário do polo'),
  (gen_random_uuid(), 'daily_log.create', 'Criar diário do polo'),
  (gen_random_uuid(), 'purchase_request.read', 'Consultar pedidos de
compra'),
  (gen_random_uuid(), 'purchase_request.create', 'Criar pedidos de compra'),
  (gen_random_uuid(), 'cabinet.read', 'Consultar gabinete'),
  (gen_random_uuid(), 'cabinet.action.read', 'Consultar ações do gabinete'),
  (gen_random_uuid(), 'cabinet.action.create', 'Criar ações do gabinete'),
  (gen_random_uuid(), 'task.read', 'Consultar tarefas'),
  (gen_random_uuid(), 'task.create', 'Criar tarefas'),
  (gen_random_uuid(), 'task.update', 'Atualizar tarefas'),
  (gen_random_uuid(), 'task.complete', 'Concluir tarefas'),
  (gen_random_uuid(), 'demand.read', 'Consultar demandas'),
  (gen_random_uuid(), 'demand.create', 'Criar demandas'),
  (gen_random_uuid(), 'demand.update', 'Atualizar demandas'),
  (gen_random_uuid(), 'demand.assign', 'Atribuir demandas'),
  (gen_random_uuid(), 'event.read', 'Consultar eventos e atividades'),
  (gen_random_uuid(), 'event.create', 'Criar eventos e atividades'),
  (gen_random_uuid(), 'event.update', 'Atualizar eventos e atividades'),
  (gen_random_uuid(), 'dashboard.admin.read', 'Consultar dashboard
administrativo'),
  (gen_random_uuid(), 'dashboard.vereador.read', 'Consultar dashboard do
vereador'),
  (gen_random_uuid(), 'dashboard.polo.read', 'Consultar dashboard do polo'),

```

```

    (gen_random_uuid(), 'dashboard.cabinet.read', 'Consultar dashboard do
gabinete'),
    (gen_random_uuid(), 'geo.read', 'Consultar camadas geográficas'),
    (gen_random_uuid(), 'geo.manage', 'Gerir camadas geográficas'),
    (gen_random_uuid(), 'report.read', 'Consultar relatórios'),
    (gen_random_uuid(), 'report.export', 'Exportar relatórios'),
    (gen_random_uuid(), 'audit.read', 'Consultar auditoria'),
    (gen_random_uuid(), 'privacy.read', 'Consultar solicitações de
privacidade'),
    (gen_random_uuid(), 'privacy.process', 'Processar solicitações de
privacidade');

```

4.1 Exemplo canônico de vinculação role-permission

```

insert into iam.role_permissions (id, role_id, permission_id)
select gen_random_uuid(), r.id, p.id
from iam.roles r
join iam.permissions p on p.code in (

    'user.read', 'user.create', 'user.update', 'user.manage_roles', 'user.manage_scopes',
    'organization.read', 'organization.create', 'organization.update',
    'vereador.read', 'vereador.create', 'vereador.update',
    'team.read', 'team.create', 'team.update',
    'person.read', 'person.create', 'person.update', 'person.link',
    'consent.read', 'consent.create', 'consent.revoke',

    'capture.read', 'capture.create', 'capture.classify', 'capture.forward', 'capture.convert',
    'polo.read', 'polo.create', 'polo.update', 'polo.manage_beneficiary',
    'modality.read', 'modality.create', 'modality.update',

    'attendance.read', 'attendance.create', 'occurrence.read', 'occurrence.create',

    'daily_log.read', 'daily_log.create', 'purchase_request.read', 'purchase_request.create',
    'cabinet.read', 'cabinet.action.read', 'cabinet.action.create',
    'task.read', 'task.create', 'task.update', 'task.complete',
    'demand.read', 'demand.create', 'demand.update', 'demand.assign',
    'event.read', 'event.create', 'event.update',

    'dashboard.admin.read', 'dashboard.vereador.read', 'dashboard.polo.read', 'dashboard.cabinet.read',

    'geo.read', 'geo.manage', 'report.read', 'report.export', 'audit.read', 'privacy.read', 'privacy.pro
)
where r.code = 'ADM_GERAL_REVISA';

```

5. Primeiros arquivos canônicos do monorepo por domínio

5.1 apps/api/app/core/database.py

```
from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase, sessionmaker

from app.core.settings import settings

class Base(DeclarativeBase):
    pass

engine = create_engine(settings.database_url, future=True,
pool_pre_ping=True)
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False,
future=True)

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

5.2 apps/api/app/core/settings.py

```
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    database_url: str
    jwt_secret_key: str
    jwt_refresh_secret_key: str
    api_v1_prefix: str = "/api/v1"

    class Config:
        env_file = ".env"
        extra = "ignore"

settings = Settings()
```

5.3 apps/api/app/core/security.py

```
from datetime import datetime, timedelta, timezone
import jwt
from passlib.context import CryptContext

from app.core.settings import settings

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def hash_password(password: str) -> str:
    return pwd_context.hash(password)

def verify_password(password: str, password_hash: str) -> bool:
    return pwd_context.verify(password, password_hash)

def create_access_token(subject: str) -> str:
    now = datetime.now(timezone.utc)
    payload = {"sub": subject, "type": "access", "iat": now, "exp": now +
timedelta(minutes=30)}
    return jwt.encode(payload, settings.jwt_secret_key, algorithm="HS256")
```

5.4 apps/api/app/api/deps/auth.py

```
from fastapi import Depends, HTTPException, status

def get_current_user():
    # stub canônico inicial
    user = type("CurrentUser", (), {
        "id": None,
        "is_global_admin": True,
        "allowed_polo_ids": set(),
    })()
    if user is None:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
detail="Unauthorized")
    return user
```

5.5 apps/api/app/shared/audit.py

```
from sqlalchemy.orm import Session
from sqlalchemy import text
```

```

def write_audit_log(
    db: Session,
    *,
    user_id,
    action: str,
    entity_schema: str,
    entity_name: str,
    entity_id,
    old_values_json=None,
    new_values_json=None,
):
    stmt = text(
        """
        insert into governance.audit_logs (
            user_id, action, entity_schema, entity_name, entity_id,
old_values_json, new_values_json
        ) values (
            :user_id, :action, :entity_schema, :entity_name, :entity_id,
cast(:old_values_json as jsonb), cast(:new_values_json as jsonb)
        )
        """
    )
    db.execute(
        stmt,
        {
            "user_id": user_id,
            "action": action,
            "entity_schema": entity_schema,
            "entity_name": entity_name,
            "entity_id": entity_id,
            "old_values_json": old_values_json,
            "new_values_json": new_values_json,
        },
    )

```

5.6 apps/api/app/domain/iam/policy.py

```

class PermissionPolicy:
    @staticmethod
    def has_permission(current_user, permission_code: str) -> bool:
        return permission_code in getattr(current_user, "permissions",
set()) or getattr(current_user, "is_global_admin", False)

```

5.7 apps/api/app/api/v1/routers/users.py

```

from fastapi import APIRouter

```

```

router = APIRouter()

@router.get("")
def list_users() -> list[dict]:
    return []

@router.post("", status_code=201)
def create_user() -> dict:
    return {"status": "created"}

```

5.8 apps/api/app/domain/core/models.py

```

from datetime import datetime, date
from uuid import uuid4

from sqlalchemy import Boolean, Date, DateTime, ForeignKey, String, Text,
func
from sqlalchemy.dialects.postgresql import UUID
from sqlalchemy.orm import Mapped, mapped_column

from app.core.database import Base

class Organization(Base):
    __tablename__ = "organizations"
    __table_args__ = {"schema": "core"}

    id: Mapped[str] = mapped_column(UUID(as_uuid=True), primary_key=True,
default=uuid4)
    type: Mapped[str] = mapped_column(String(30), nullable=False)
    name: Mapped[str] = mapped_column(String(255), nullable=False)
    legal_name: Mapped[str | None] = mapped_column(String(255))
    document_number: Mapped[str | None] = mapped_column(String(30))
    parent_organization_id: Mapped[str | None] =
mapped_column(UUID(as_uuid=True), ForeignKey("core.organizations.id"))
    active: Mapped[bool] = mapped_column(Boolean, default=True,
nullable=False)
    created_at: Mapped[datetime] = mapped_column(DateTime,
server_default=func.now(), nullable=False)
    updated_at: Mapped[datetime] = mapped_column(DateTime,
server_default=func.now(), nullable=False)

class Person(Base):
    __tablename__ = "persons"
    __table_args__ = {"schema": "core"}

    id: Mapped[str] = mapped_column(UUID(as_uuid=True), primary_key=True,

```

```

default=uuid4)
    full_name: Mapped[str] = mapped_column(String(255), nullable=False)
    social_name: Mapped[str | None] = mapped_column(String(255))
    cpf: Mapped[str | None] = mapped_column(String(20))
    birth_date: Mapped[date | None] = mapped_column(Date)
    phone: Mapped[str | None] = mapped_column(String(30))
    secondary_phone: Mapped[str | None] = mapped_column(String(30))
    email: Mapped[str | None] = mapped_column(String(180))
    gender: Mapped[str | None] = mapped_column(String(30))
    notes: Mapped[str | None] = mapped_column(Text)
    created_at: Mapped[datetime] = mapped_column(DateTime,
server_default=func.now(), nullable=False)
    updated_at: Mapped[datetime] = mapped_column(DateTime,
server_default=func.now(), nullable=False)

```

5.9 apps/api/app/domain/core/schemas.py

```

from pydantic import BaseModel
from uuid import UUID
from datetime import date

class PersonCreate(BaseModel):
    full_name: str
    social_name: str | None = None
    cpf: str | None = None
    birth_date: date | None = None
    phone: str | None = None
    secondary_phone: str | None = None
    email: str | None = None
    gender: str | None = None
    notes: str | None = None

class PersonOut(BaseModel):
    id: UUID
    full_name: str
    social_name: str | None = None
    cpf: str | None = None
    birth_date: date | None = None
    phone: str | None = None
    email: str | None = None

    model_config = {"from_attributes": True}

```

5.10 apps/api/app/domain/core/repository.py

```
from sqlalchemy import select
from sqlalchemy.orm import Session

from app.domain.core.models import Person

class CoreRepository:
    def __init__(self, db: Session):
        self.db = db

    def list_persons(self):
        return self.db.execute(select(Person)).scalars().all()

    def create_person(self, entity: Person) -> Person:
        self.db.add(entity)
        self.db.flush()
        self.db.refresh(entity)
        return entity
```

5.11 apps/api/app/domain/core/service.py

```
from app.domain.core.models import Person
from app.domain.core.repository import CoreRepository
from app.domain.core.schemas import PersonCreate

class CoreService:
    def __init__(self, repo: CoreRepository):
        self.repo = repo

    def list_persons(self):
        return self.repo.list_persons()

    def create_person(self, payload: PersonCreate):
        entity = Person(**payload.model_dump())
        return self.repo.create_person(entity)
```

5.12 apps/api/app/api/v1/routers/persons.py

```
from fastapi import APIRouter, Depends
from sqlalchemy.orm import Session

from app.core.database import get_db
from app.domain.core.repository import CoreRepository
from app.domain.core.schemas import PersonCreate, PersonOut
```



```

from app.domain.core.service import CoreService

router = APIRouter()

@router.get("", response_model=list[PersonOut])
def list_persons(db: Session = Depends(get_db)):
    return CoreService(CoreRepository(db)).list_persons()

@router.post("", response_model=PersonOut, status_code=201)
def create_person(payload: PersonCreate, db: Session = Depends(get_db)):
    return CoreService(CoreRepository(db)).create_person(payload)

```

6. Organização recomendada de arquivos iniciais a materializar primeiro

Sprint técnico imediato

Obrigatórios

- apps/api/alembic/versions/0004_create_polo_schema.py
- apps/api/alembic/versions/0005_create_events_schema.py
- apps/api/alembic/versions/0006_create_workflow_schema.py
- apps/api/alembic/versions/0007_create_governance_schema.py
- apps/api/alembic/versions/0008_create_analytics_schema.py
- apps/api/app/domain/core/models.py
- apps/api/app/domain/core/schemas.py
- apps/api/app/domain/core/repository.py
- apps/api/app/domain/core/service.py
- apps/api/app/domain/territory/models.py
- apps/api/app/domain/territory/schemas.py
- apps/api/app/domain/territory/repository.py
- apps/api/app/domain/territory/service.py
- apps/api/app/domain/polo/models.py
- apps/api/app/domain/polo/schemas.py
- apps/api/app/domain/polo/repository.py
- apps/api/app/domain/polo/service.py
- apps/api/app/domain/workflow/models.py
- apps/api/app/domain/workflow/schemas.py
- apps/api/app/domain/workflow/repository.py
- apps/api/app/domain/workflow/service.py
- apps/api/app/domain/analytics/repository.py
- apps/api/app/domain/analytics/service.py
- apps/api/app/api/v1/routers/persons.py
- apps/api/app/api/v1/routers/contacts_capture.py

- `apps/api/app/api/v1/routers/polos.py`
- `apps/api/app/api/v1/routers/tasks.py`
- `apps/api/app/api/v1/routers/dashboards.py`
- `database/seeds/permissions_seed.sql`

Em seguida

- policies específicas por domínio
 - auditoria automática por decorator ou service helper
 - testes de integração para create/list por domínio
 - autenticação JWT real no `get_current_user`
-

7. Fechamento

Este pack já representa uma base operacional real para o time iniciar materialização do backend e do monorepo sem rediscutir estrutura.

Ele entrega:

- migrations restantes em formato Alembic
- stubs reais coerentes com os domínios
- seed completo de permissões
- arquivos canônicos mínimos para o backend evoluir por domínio

A evolução mais correta daqui é transformar esse conteúdo em arquivos físicos do repositório e fechar logo depois:

- testes de integração mínimos
- policies por escopo
- auditoria central por service helper
- bindings entre permissões e dependências FastAPI